

VNUHCM - UNIVERSITY OF SCIENCE FACULTY OF INFORMATION TECHNOLOGY



CS486 COURSE REPORT – PHASE 2

Campus Space Management System

Members of group 05:	MSSV:
Phạm Hữu Nam	24125015
Cao Quảng Hưng	24125010
Nguyễn Hữu Phước	24125018
Trần Đình Quốc Thắng	24125079

TP. Hồ Chí Minh, 08/2026

Table of Contents

1	LLM Usage & Agent Improvement Process	4
1.1	LLM Models Used	4
1.2	Evaluation and Improvement Cycle	4
2	Requirement Change Summary	6
2.1	Changes from Phase 1	6
2.2	Affected Data and Business Rules	6
3	Updated Design	7
3.1	Updated Entity–Relationship Diagram	7
3.2	Updated Relational Schema	8
3.3	Design Justification	10
4	Schema Migration	10
4.1	Migration Approach	10
4.2	Data Preservation and Backfill	11
4.3	Verification	12
5	Concurrency Design & Implementation	12
5.1	Identified Concurrency Conflicts	12
5.2	Cause and Chosen Control	14
5.2.1	Cause	14
5.2.2	Application-Lock Protocol	14
5.2.3	Invariant Revalidation and Failure Handling	14
5.2.4	Design Rationale and Trade-offs	14
5.3	Concurrent Test Results	15
5.3.1	Test Architecture and Methodology	15
5.3.2	Evaluation Criteria	15
5.3.3	Scenario Analysis and Results	16
5.3.4	Comparison Results & Invariant Verification	18
6	Sample Data Generation	19
6.1	Scale and Time Coverage	19
6.2	Generation Strategy	19
6.3	Exceptional Cases	19
7	Analytical Queries	20
7.1	Approved Booking Hours per Space (Report 1)	21
7.2	Bookings by Weekday and Hour (Report 2)	22
7.3	Available Spaces with Required Facilities (Report 3)	23
7.4	Approved Bookings Affected by Escalation (Report 4)	25
7.5	Work Assignment	28
8	Indexing & Query Tuning	28
8.1	Benchmark Setup	28
8.2	Selected Workloads and Plan Evidence	29
8.3	Before-and-After Results	29
8.4	Final Recommendation	30
9	Normalization Validation	30

9.1	Functional Dependencies	30
9.2	Third Normal Form Validation	31
10	Conclusion	32

1 LLM Usage & Agent Improvement Process

1.1 LLM Models Used

- **Big Pickle:** Evaluated via the OpenCode Zen platform, this model is specifically optimized for automated coding agents. Featuring an extensive context window of approximately 200k tokens, it serves as an ideal tool for rapid initial prototyping. However, its primary limitation lies in the absence of advanced reasoning features.
- **DeepSeek V4 Flash Free:** Accessed through the OpenCode platform, this model was deployed to facilitate technical drafting, content refinement, and structural editing, such as database design documentation. It demonstrates high efficiency in generating structured explanations and concise academic prose, though it necessitates human oversight to ensure rigorous factual accuracy and alignment with project specifications.

1.2 Evaluation and Improvement Cycle

The Phase 2 cycle was evaluated task by task against the Phase 2 requirement document, the latest task artifact, and the newest trajectory or evaluation log. The shared **AGENTS.md** policy stayed stable; most improvements were made in task-specific skill notes, validation steps, and review checklists. Table 1 summarizes the evidence, issues, and improvements recorded for each Phase 2 task.

Task	Evaluation evidence	Issue found	Improvement made
08	Task 08 artifact, Phase 2 description, and Progress memory.	The analysis exposed unresolved design questions and concurrency conflicts, but no final design choice yet.	Kept U1–U5 and K1–K5 as explicit follow-up gates so later tasks could resolve them instead of guessing.
09	Task 09 trajectory, revision log in <code>outputs/09</code> , and <code>docs/design-decisions.md</code> .	The v2.5 policy was too broad, especially the temporary duration-cap gate.	Rebased the design on data-driven purpose membership, then removed the duration cap in v2.6 and pushed the checks-1-to-5 contract downstream.
10	Scratch compile cycle, rev5 migration log, and Task 10 trajectory.	Earlier revisions still carried stale cap logic and needed stricter audit/recompute handling.	Removed <code>max_hours</code> from migration and rollback, switched audit attribution to <code>SESSION_CONTEXT</code> , and added idempotent rerun plus post-rollback count checks.
11	Static verification log plus the final Task 11 rev 3.4 design and earlier trajectories.	K5 was initially left open, and review found stale Q5 wording plus BR2 gate labeling drift.	Expanded the design to four write entry points, added ticket creation as a locked path, and tightened the review scan for cross-reference and wording drift.
12	Rev 3 compile/smoke log and the aligned Task 11 v3.4 contract.	The implementation still referenced removed cap logic and stale preflight checks.	Rebased the procedure on v2.6, removed the duration gate, and kept compile, smoke, and idempotency as mandatory validation steps.
13	Static PASS log, runner script, invariant audit, and generated scenario set.	The baseline-vs-controlled comparison needed to be explicit, and a few scenario timings had to be corrected.	Required the full baseline twin set, explicit error-code assertions, invariant auditing, and cleanup proof in the review checklist.
14	Live execution/verification log plus the final generator output and manifest.	Smoke testing, performance, calendar span, sqlcmd variable handling, and advisory-ack generation all exposed defects.	Added smoke mode, precomputed day weights, extended the calendar to four academic years, fixed the load script, and re-verified reproducibility and invariants.
15	Plan-summary artifacts, before/after ShowPlan XML captures, and the revised Task 15 report.	Execution-plan evidence was incomplete at first, and the Q2 recommendation wording overstated the measured result.	Added the plan-evidence subsection, captured plan XML before and after indexing, and aligned the recommendation to the measured gains.
16	SQL Server execution log for the final Q1–Q5 batch.	Q2 initially tested only one required facility, and direct <code>SELECT</code> queries required copy-paste for reuse.	Added multiple required facilities, reran Q1–Q5, and converted the queries into reusable stored procedures.

Table 1: Phase 2 agent evaluation and improvement log

2 Requirement Change Summary

After the one-semester pilot, the Facility Manager revised one maintenance rule and introduced new operating conditions. This section summarizes the results of `08-requirement-change-analysis`. Three broad changes were identified - maintenance impact levels (C1), concurrent booking and approval (C2), and new reporting needs (C3) - and are distinguished below from the rules retained from Phase 1.

2.1 Changes from Phase 1

C1 - Maintenance impact levels. A maintenance record now carries an impact level, `advisory` or `out-of-service` (new rule NR1). Phase 1 treated *every* active maintenance as blocking a booking; Phase 2 blocks a booking only when the overlapping maintenance is `out-of-service` (refined BR4). `advisory` maintenance leaves the space bookable, but the requester must be notified of all active advisories at booking time and the system must record the acknowledgement with the booking (NR2). A space may carry several concurrent maintenance records of different levels, a level may be escalated or downgraded while the record is open (NR3), and any already-approved bookings affected by an escalation must remain queryable so that staff can contact those requesters (NR4).

C2 - Instant booking and concurrency. Near-simultaneous requests are expected at the start of each semester. For eligible space types, requests that satisfy the usage policy are approved automatically at submission (instant booking, NR5); all other requests keep the staff-approval workflow. Both pathways must satisfy the Phase 1 no-overlap invariant (BR1): two approved bookings may never use the same space during overlapping periods, even when several users and staff operate concurrently (NR6). Task 08 identified five concurrency conflicts (K1–K5) that can violate this invariant or expose a stale availability reading; they are analyzed and resolved in Section 5.

C3 - New reporting needs. With the accumulated booking and maintenance history, the Facility Manager needs four reports: total approved booking hours per space per semester, approved bookings by weekday and hour per semester, a room finder that lists spaces satisfying capacity and facility requirements within a time period, and the approved bookings already affected when a maintenance record is escalated to `out-of-service`. The Phase 1 rules BR1, BR3, BR6, and BR7 remain unchanged and are not altered by C1–C3.

2.2 Affected Data and Business Rules

The most affected entities are **maintenance** (impact level and escalation history), **bookings** (advisory acknowledgement), and **booking_approvals** and **spaces** (automatic approval pathway and availability hint). The entities **departments**, **users**, **facilities**, **space_facilities**, and **booking_sessions** are unchanged; the room-finder report reads **facilities** and **space_facilities** more heavily, but their shape is unchanged. The affected relationships are Spaces–Maintenance (*R1/R7*), whose blocking semantics become level-dependent, and the new conceptual dependency between a booking and the advisories it was informed about, which Task 09 resolves as an acknowledgement table. Table 2 traces each change to the design and implementation artifact that realizes it.

Change	Affected data	Business rule	Implemented in
C1	maintenance; booking_advisory_acknowledgement	BR2/BR4 refined; NR1–NR4	Task 09, §A; Task 10; Task 16, Q5
C2	bookings; booking_approvals; users; spaces	BR1 across both pathways; NR5, NR6	Task 09, §B; Tasks 11–13
C3	bookings; spaces; space_facilities	BR14 reporting set	Task 16; Task 15 tuning report

Table 2: Traceability of Phase 2 requirement changes

3 Updated Design

This section presents the results of 09-updated-erd-and-logical-design. The Phase 2 extension was designed in three scoped areas on top of the Phase 1 baseline: (1) maintenance impact levels and advisory acknowledgements, (2) concurrent and instant booking and approval, and (3) the new analytical reporting needs. Areas 1 and 2 introduce schema changes; Area 3 is fully answerable as queries over the updated schema and adds no table.

3.1 Updated Entity–Relationship Diagram

Figure 1 presents the updated entity–relationship diagram for the Phase 2 design.

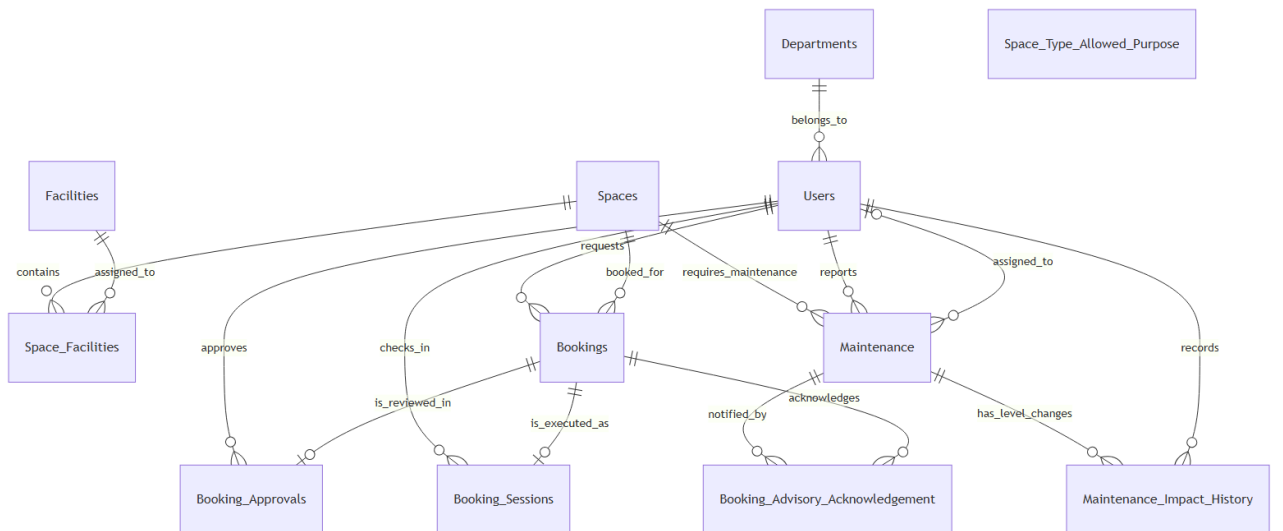


Figure 1: Updated ERD for the Campus Space Management System

Area 1 extends the existing `maintenance` entity with the `impact_level` attribute and adds two new entities. `maintenance_impact_history` records every escalation or downgrade of an open record (NR3), and `booking_advisory_acknowledgement` resolves the new many-to-many dependency between a booking and the active advisories it was informed about (NR2) into an associative table with one row per (booking, advisory) pair. Area 2 adds the reference entity `space_type_allowed_purpose`, which makes instant-booking eligibility data-defined, while the instant/staff pathway of a booking is derived: an approval is instant if and only if `approver_id` = -1. The new and changed relationships are listed in Table 3.

Relationship	Cardinality	Participation	Meaning
<code>maintenance</code> → <code>maintenance_impact_history</code>	1:N	History: total	Every level change belongs to exactly one maintenance record
<code>users</code> → <code>maintenance_impact_history</code>	1:N	History: total	Each change is recorded by exactly one user
<code>maintenance</code> → <code>booking_advisory_acknowledgement</code>	1:N	Acknowledgement: total	Each acknowledgement refers to exactly one maintenance record
<code>bookings</code> → <code>booking_advisory_acknowledgement</code>	1:N	Acknowledgement: total	Each acknowledgement is attached to exactly one booking

Table 3: New and changed relationships in the updated ERD

3.2 Updated Relational Schema

Area 1 changes `maintenance` by adding `impact_level`, and creates `maintenance_impact_history` and `booking_advisory_acknowledgement`. Area 2 removes the unused free-text column `spaces.usage_policy` and creates `space_type_allowed_purpose`. The remaining changes are data-only (the reserved system user) or derived (the instant origin). Table 4 summarizes every new or changed table with its keys, constraints, and the rule that motivates it.

Table	Change	Key or constraint changes	Reason
<code>maintenance</code>	Column added	<code>impact_level</code> VARCHAR(50) NOT NULL; CHECK permits <code>advisory</code> or <code>out-of-service</code> ; default <code>out-of-service</code>	NR1; the default preserves Phase 1 blocking for legacy rows on migration
<code>maintenance_impact_history</code>	New table	PK <code>history_id</code> ; FK <code>maintenance_id</code> (CASCADE); FK <code>changed_by</code> ; CHECK prior $\neq$ new level	NR3; <code>changed_at</code> anchors the NR4 escalation-time report
<code>booking_advisory_acknowledgement</code>	New table	PK <code>ack_id</code> ; UNIQUE (<code>booking_id</code> , <code>maintenance_id</code>); FKs to <code>bookings</code> and <code>maintenance</code>	NR2; at most one acknowledgement per (booking, advisory) pair
<code>booking_approvals</code>	No column change	Origin derived: <code>approver_id</code> = -1 means instant; otherwise staff-approved	NR5; the instant/staff pathway is derived from the reserved system approver
<code>users</code>	Data change only	Reserved system row <code>user_id</code> = -1 with role <code>facility_manager</code>	approver identity for auto-approval; keeps BR6/BR15 intact
<code>spaces</code>	Column removed	<code>usage_policy</code> dropped	D9; the free-text policy was never enforced and collided with the data-driven usage-policy test
<code>space_type_allowed_purpose</code>	New table	Composite PK (<code>space_type</code> , <code>purpose</code>); CHECK mirrors the existing domains	D11; data-defined instant eligibility for the four approved space types

Table 4: Phase 2 relational schema changes

3.3 Design Justification

The design keeps the Phase 2 extension additive and data-preserving. New columns and tables extend the frozen Phase 1 schema without rebuilding any table, and the **out-of-service** default on **maintenance.impact_level** guarantees that migrated legacy records keep their Phase 1 blocking behavior until staff explicitly downgrade them.

Acknowledgements are stored as one row per (booking, advisory) pair. This makes the NR2 completeness gate checkable at approval time - an approval is rejected if any overlapping active advisory lacks an acknowledgement - and makes the NR4 report of escalated bookings derivable by joining acknowledgements with the maintenance impact history; notifying the affected requesters is a follow-up action taken by staff from that report.

The instant/staff pathway is a derived attribute of **booking_approvals**: an approval is instant if and only if **approver_id** = -1, the reserved system approver, and staff-approved otherwise. Deriving the pathway instead of storing it introduces no extra functional dependency on **booking_approvals**, which therefore remains in BCNF (Section 9). Instant eligibility is data-defined in **space_type_allowed_purpose**, so changing the eligible set or the allowed purposes is a configuration change rather than a code change.

Finally, two concerns are deliberately deferred rather than modeled in the schema: concurrency safety around the no-overlap invariant is enforced by the Task 11 application-lock protocol on top of the unchanged **uq_bookings_active_overlap** object, and the four reports are derived queries over the updated schema. Neither requires additional ERD/logical schema, which keeps the design compatible with the existing Phase 1 data.

4 Schema Migration

Phase 2 extends the frozen Phase 1 schema with maintenance impact levels, advisory acknowledgements, and data-driven instant booking. Since the database already contained booking and maintenance data, the upgrade was implemented as a data-preserving T-SQL delta in **10-schema-migration.sql**. The script upgrades the Task 05 baseline, **05-db-definition.sql**, to the approved Task 09 design. The companion script **10-schema-migration-rollback.sql** reverses the upgrade when required.

4.1 Migration Approach

The migration is primarily additive: it introduces new columns, tables, indexes, and triggers without rebuilding or renaming any Phase 1 table. The only removed element is **spaces.usage_policy**, an unused free-text column superseded by the Task 09 reference-table design. No rule, index, or trigger depended on this column.

The complete schema delta is summarized in Table 5.

Schema component	Phase 2 implementation
Maintenance impact	Add <code>maintenance.impact_level</code> as NOT NULL, defaulting to 'out-of-service'. A check constraint permits only out-of-service and advisory.
Impact history	Add <code>maintenance_impact_history</code> to record every maintenance impact-level change.
Advisory acknowledgement	Add <code>booking_advisory_acknowledgement</code> . A composite unique constraint permits one acknowledgement per booking–advisory pair.
Instant-booking policy	Add <code>space_type_allowed_purpose</code> and remove the superseded <code>spaces.usage_policy</code> column.
Trigger logic	Restrict booking blocks to out-of-service maintenance; record impact history; validate acknowledgement ownership; maintain modification timestamps; and recompute space status.
Configuration data	Seed one reserved system approver and 18 allowed-purpose pairs for the four approved space types.

Table 5: Phase 2 schema migration delta

The revised booking and maintenance triggers block overlaps only when a space has out-of-service maintenance; advisory maintenance produces a warning but does not make the space unavailable. Additional triggers record impact-level changes, verify that acknowledgements are submitted by the booking owner, maintain last-modified timestamps, and recompute `spaces.current_status`. Status is derived using the approved priority order, avoiding duplicated state.

The migration is both transactional and repeatable. All operations execute in a single transaction, so a failed or interrupted run leaves the database unchanged. Existence checks guard object creation and seed insertion, allowing the script to be rerun without creating duplicate objects or rows. The rollback script restores the three Phase 1 trigger definitions and removes every column, table, index, trigger, and seed row introduced by the migration, returning the schema to the Task 05 baseline.

4.2 Data Preservation and Backfill

No existing booking or maintenance row is deleted or assigned a new identity. The only backfill sets the new maintenance impact level to 'out-of-service'. This preserves Phase 1 behavior because every existing active maintenance record continues to block overlapping bookings.

After the schema changes, `spaces.current_status` is recomputed using the priority order: retired, temporarily closed, out-of-service maintenance, live session, and available. Existing manual `retired` and `temporarily_closed` overrides are preserved.

The migration also inserts the reserved system approver used to identify derived instant approvals and the 18 (`space_type`, `purpose`) pairs that define instant-booking eligibility for the four approved space types. Guarded inserts ensure that each row is created only once and do not disturb existing identity values.

4.3 Verification

Verification was performed on a scratch SQL Server 2022 database. The Task 05 DDL and sample data were loaded first, followed by the migration, a second run of the same migration, and the rollback. Validation queries confirmed that the new schema objects had the approved definitions, every maintenance row had an impact level, and the system approver and all 18 allowed-purpose pairs existed exactly once. The second migration run completed without errors or duplicates, confirming repeatability.

Transactional smoke tests covered the main behavioral rules. Advisory maintenance allowed overlapping bookings, whereas out-of-service maintenance continued to block them. Duplicate acknowledgement of the same advisory was rejected, as was approval without the required acknowledgement. Each test was rolled back after execution. Finally, the rollback script restored the baseline row counts exactly, confirming that the migration and reversal caused no data loss.

5 Concurrency Design & Implementation

Phase 2 introduces a high-concurrency operating condition: at the start of a semester, many users may request popular spaces for overlapping periods within a short interval. Eligible requests are confirmed immediately through instant booking, whereas other requests follow the staff-approval workflow. Since both pathways can confirm the same space–time interval concurrently, the database must enforce NR 6: *two approved bookings must never use the same space during overlapping periods, regardless of how they were created*.

Task 08 identified five concurrency conflicts (K1–K5) that can violate this invariant or expose stale availability. The solution was specified in `11-concurrency-design`, implemented in `12-concurrency-implementation.sql`, and verified through deterministic two-session tests in `13-concurrency-tests`. It serializes the four write paths that can change the confirmed state of a space–time interval: instant submission, staff approval, maintenance escalation or downgrade, and maintenance-ticket creation. Each path uses the same logical resource for a space and rechecks the relevant invariants immediately before writing.

5.1 Identified Concurrency Conflicts

The primary conflict is the K1 *read-check-write race*. Two concurrent requests for the same space and period can both observe the interval as available and then confirm their bookings before either transaction sees the other’s result. Table 6 illustrates this race for two instant-booking sessions targeting space S_1 from 10:00 to 12:00.

Step	Session A: instant submission	Session B: instant submission
1	Check S_1 : no confirmed overlap or blocking maintenance.	
2		Check S_1 : no confirmed overlap or blocking maintenance.
3	Insert a pending booking and its automatic approval (<code>approver_id = -1</code>).	
4		Insert a pending booking and its automatic approval; BR1 and BR4 checks see no committed competitor.
5	Commit.	
6		Commit.

Table 6: Concurrent instant-booking race for the same space and period

At step 4, session A’s write remains uncommitted and is therefore not visible to session B under the default READ COMMITTED isolation level. Both transactions pass the availability rule (BR1) and maintenance rule (BR4), then commit overlapping approved bookings on S_1 , violating BR1 and NR6.

The interval between the availability check and the confirmation write is the *race window*. Because no lock spans this interval, neither existing backstop fully closes it. The trigger **trg_bookings_prevent_overlap** cannot validate against a competing row that is not yet visible, while the filtered unique index **uq_bookings_active_overlap** prevents only identical start times. It does not reject intersecting intervals with different start times, such as 10:00–12:00 and 11:00–13:00.

The same check–write race occurs across other booking and maintenance paths. K2 covers instant submission racing with staff approval. K3 occurs when a maintenance record is escalated to **out-of-service** during an in-flight confirmation, while K5 covers insertion of a new maintenance ticket whose default impact is **out-of-service**. K4 is a read-side conflict in which room-finder availability becomes stale before the user acts. Table 7 summarizes the complete conflict inventory.

ID	Concurrent scenario	Affected rules	Path
K1	Two requests read the same interval as free and both confirm.	BR1, NR6	Booking insertion
K2	Instant submission races with staff approval.	BR1, NR6	Cross-pathway confirmation
K3	Maintenance is escalated while a booking confirmation is in flight.	BR4, NR4	Maintenance update
K4	Room-finder availability becomes stale before confirmation.	BR1, NR4, NR6	Read-only availability
K5	An out-of-service ticket is inserted during confirmation.	BR4, NR6	Maintenance insertion

Table 7: Concurrency conflicts identified in the Task 08 analysis

K1, K2, K3, and K5 share the same root cause: concurrent transactions perform a check and a conflicting write on the same space–time interval without a common serialization point. K4 cannot by itself corrupt the database, but its stale result must not be trusted during

confirmation. The implemented control therefore serializes the check–recheck–write sequence per space and validates availability again inside the protected transaction. Consequently, two conflicting confirmations cannot both commit.

5.2 Cause and Chosen Control

5.2.1 Cause

The race occurs under the default READ COMMITTED isolation level because a transaction cannot observe another transaction’s uncommitted rows. Two writers can therefore validate the same availability state before either commits. The existing triggers execute within their writer’s transaction and cannot validate against an invisible competitor, while the filtered unique index rejects only identical start times. Since no common serialization point spans the check–write interval, all confirmation-related write paths require one.

5.2.2 Application-Lock Protocol

The implementation uses a transaction-owned SQL Server application lock. After starting a transaction, each write procedure calls `sys.sp_getapplock` with the following configuration:

- Resource: `space_booking:<space_id>`;
- Mode: **Exclusive**;
- Owner: **Transaction**;
- Timeout: 5 seconds.

The lock is released automatically when the transaction commits or rolls back. The procedures `usp_booking_instant_submit`, `usp_booking_approve`, `usp_maintenance_set_impact_level`, and `usp_maintenance_report` request the same resource for a given space. This serializes instant approvals, staff approvals, and booking–maintenance interactions through one protocol. Maintenance reporting acquires the lock only when the new ticket begins as **out-of-service**; an advisory ticket does not block the interval and instead creates an acknowledgement requirement.

5.2.3 Invariant Revalidation and Failure Handling

The lock protects the critical section, but correctness depends on revalidation inside it. Immediately before writing, each procedure rereads its target data and checks confirmed-booking overlap (BR1), out-of-service maintenance overlap (BR4), space status (BR2), capacity (BR3), and acknowledgement completeness (NR2). A failed check rolls back the transaction and returns a deterministic result code. Existing triggers and the filtered unique index remain as defense-in-depth controls.

Lock-acquisition failures are returned to the caller rather than silently retried. A timeout returns 51005, a cancelled request returns 51006, and both are retryable. A deadlock victim returns 51007; in this case, the caller must restart the entire transaction.

5.2.4 Design Rationale and Trade-offs

The application lock is valid under READ COMMITTED and requires no schema change. In contrast, SERIALIZABLE key-range locking or UPDLOCK / HOLDLOCK range reads are reliable only when every writer locks the same predicate range. The four write paths use

different interval operations, making this approach plan-sensitive and vulnerable to a range-boundary error that could reopen K3 or K5. Optimistic concurrency would require version columns excluded from the approved design, while triggers alone cannot serialize concurrent writers under READ COMMITTED.

The lock resource is keyed by space rather than by space and day. A daily key would require a multi-day booking to acquire several locks in a fixed order, creating additional lock-ordering and cross-midnight risks. The per-space key avoids these cases but serializes all writers for a popular space. This cost is acceptable for the expected workload: Task 14 generates more than 100,000 bookings across three academic years, corresponding to tens of writes per day for the busiest space, and each critical section completes well within the five-second timeout. If production monitoring later identifies material contention, a per-day resource key remains the documented tuning option.

5.3 Concurrent Test Results

To evaluate the effectiveness and reliability of the Task 12 concurrency control mechanisms, a comprehensive **Comparison Suite** was developed in `outputs/13-concurrency-tests-G05`. Each conflict scenario is delivered as a pair:

- **Baseline (Uncontrolled - Raw DML):** Direct T-SQL execution (INSERT, UPDATE) without application-level locks, operating under READ_COMMITTED_SNAPSHOT ON (RCSI) mode. Under RCSI, Phase 1 triggers read committed row versions from `tempdb` without taking Shared Locks. When two transactions execute concurrently before either commits, Phase 1 triggers are bypassed, committing overlapping rows to disk ($Q_{BR1} \geq 1$), or throwing unhandled engine exceptions (Msg 50000) when a competitor commits first.
- **Controlled (Task 12 Stored Procedures):** All write operations route strictly through Task 12 entry points (`usp_booking_instant_submit`, `usp_booking_approve`, `usp_maintenance_report`, `usp_maintenance_set_impact_level`). Each procedure acquires an Exclusive SQL Server Application Lock (`sys.sp_getapplock` on `space_booking:<space_id>`) with a 5-second timeout, ensuring 100% data invariant preservation ($Q_{BR1} = 0$, $Q_{NR6} = 0$) and structured business result codes (51001–51009).

5.3.1 Test Architecture and Methodology

- **Dual-Session Parallel Execution:** An automated test runner orchestrates two independent database client sessions (Session A and Session B) concurrently in parallel within the same millisecond to force genuine lock contention and induce race condition windows under active workload.
- **Isolated Test Environment:** An automated setup script seeds an isolated test workspace featuring dedicated departments, accounts, and space instances with booking time windows scheduled far into the future ($\geq +600$ days). This guarantees zero collision or data corruption with existing production sample data.
- **Idempotent Lifecycle Management:** An automated teardown script executes prior to and following every test scenario to perform complete, foreign-key-safe data cleanup, ensuring full test repeatability and isolation across test runs.

5.3.2 Evaluation Criteria

- **Baseline Verification (Vulnerability Demonstration):** A Baseline scenario passes verification by empirically exposing the systemic flaws of raw DML under concur-

rency—either through trigger bypass resulting in committed overlapping data corruption ($Q_{BR1} \geq 1$ or $Q_{NR6} \geq 1$), or via unhandled database engine exceptions (`Msg 50000 / Msg 3609`) that abort client transactions.

- **Controlled Verification (Application-Level Control):** A Controlled scenario passes verification when it satisfies three mandatory guarantees:
 1. *Data Integrity Preservation:* Zero overlapping confirmed bookings ($Q_{BR1} = 0$) and zero out-of-service maintenance conflicts ($Q_{NR6} = 0$).
 2. *Application Serialization:* The winning transaction receives `rc = 0`, while the losing transaction receives an explicit structured business status code (51003, 51002, 51005).
 3. *Zero Engine Crashes:* 0% unhandled database engine exceptions (`Msg 50000`), allowing client applications to handle responses gracefully.

5.3.3 Scenario Analysis and Results

The comparison suite evaluates nine distinct concurrency test scenarios representing the five conflict families (K1–K5) and additional operational edge cases. Rather than embedding verbose T-SQL code listings into the document body, each scenario is abstracted below by its core objective, observed baseline vulnerability, and controlled resolution:

1. Scenario 1: Concurrent Instant Submissions (b01 vs c01)

- *Objective:* Verify system behavior when two users simultaneously submit instant bookings for the exact same space and overlapping time window.
- *Baseline Result:* Under RCSI isolation, Phase 1 triggers read committed row versions from `tempdb` without taking Shared Locks. Both concurrent `INSERT` statements execute without blocking, bypassing the availability check and committing two overlapping confirmed bookings to disk ($Q_{BR1} = 1$).
- *Controlled Result:* The Exclusive Application Lock on `space_booking:<space_id>` serializes execution. Session A acquires the lock first and commits (`rc = 0`). Session B waits, rechecks availability inside the lock, detects Session A’s confirmed row, and rejects the submission with structured business code 51003 (BR1 Overlap Conflict), preserving $Q_{BR1} = 0$.

2. Scenario 2: Instant Submission vs. Staff Approval (b02 vs c02)

- *Objective:* Test cross-pathway concurrency when an instant booking submission races with a staff member approving an existing pending booking for the same interval.
- *Baseline Result:* Without application locks, a raw `INSERT` for an instant booking commits concurrently alongside a staff approval transaction, producing overlapping approved bookings on disk ($Q_{BR1} = 1$).
- *Controlled Result:* Both pathways acquire the same per-space application lock resource. Whichever procedure executes first acquires the lock and confirms its booking (`rc = 0`); the subsequent procedure revalidates availability within the lock and returns 51003, maintaining $Q_{BR1} = 0$.

3. Scenario 3: Maintenance Escalation vs. In-flight Confirmation (b03 vs c03/c03b)

- *Objective:* Evaluate protection when a staff member escalates an existing advisory maintenance record to **out-of-service** during an active booking confirmation.

- *Baseline Result:* Raw DML execution causes the competing booking submission to hit the Phase 1 trigger after the escalation commits, throwing an unhandled engine exception (Msg 50000) that aborts the client batch.
- *Controlled Result:* `usp_maintenance_set_impact_level` acquires the per-space lock before altering the impact status. The escalation completes (`rc = 0`); any concurrent or subsequent booking attempt over the out-of-service window is rejected cleanly with 51002 (BR4 Out-of-Service Conflict), ensuring $Q_{NR6} = 0$.

4. Scenario 4: Lock Timeout and Client Retry (b05 vs c05)

- *Objective:* Validate system resilience and timeout contracts when a transaction holds a space resource lock during an extended operation.
- *Baseline Result:* Raw row-level updates block competing transactions indefinitely (blocking observed for 18+ seconds), exposing clients to unbounded connection hangs.
- *Controlled Result:* `usp_maintenance_report` enforces a strict 5-second lock timeout. The competing session times out after 5 seconds, returning retryable error code 51005. A subsequent retry after the first transaction releases the lock succeeds with `rc = 0`.

5. Scenario 5: Out-of-Service Ticket Creation vs. Booking Submit (b09 vs c09)

- *Objective:* Test concurrency when a new maintenance ticket with initial **out-of-service** status is created concurrently with a booking submission.
- *Baseline Result:* Raw DML inserts result in unhandled trigger exceptions (Msg 50000) when one transaction commits before the other.
- *Controlled Result:* Maintenance ticket creation acquiring initial out-of-service impact acquires the Exclusive AppLock. The ticket creation completes (`rc = 0`), while the concurrent booking attempt is rejected cleanly with 51002, guaranteeing $Q_{NR6} = 0$.

6. Scenario 6: Dual Staff Approval Race (b10 vs c10)

- *Objective:* Test the race condition where two staff members simultaneously approve two different pending requests for overlapping intervals on the same space.
- *Baseline Result:* Under RCSI, both staff approval transactions read pending status from `tempdb` and insert approval rows without blocking, committing two overlapping approved bookings to disk ($Q_{BR1} = 1$).
- *Controlled Result:* `usp_booking_approve` serializes all staff approvals per space. The first approval succeeds (`rc = 0`); the second approval rechecks confirmed bookings inside the lock, detects the first approval, and fails with 51003, preserving $Q_{BR1} = 0$.

7. Scenario 7: Soft-Gate Fallback for Disallowed Purposes (c11)

- *Objective:* Test business logic fallback when a user submits an instant booking with an ineligible purpose (e.g., `lecture` on a `meeting_room`).
- *Controlled Result:* `usp_booking_instant_submit` validates space purpose eligibility. Instead of throwing an error or auto-approving, it inserts the request under `pending` status (`rc = 0`, `instant_accepted = 0`), routing it for manual staff review.

8. Scenario 8: Pending Fallback vs. Instant Submit Race (c12)

- *Objective:* Ensure that a pending fallback booking does not block a subsequent instant booking for the same interval, but any later approval of the pending booking is correctly rejected.

- *Controlled Result:* The instant booking successfully acquires the space lock and auto-approves (`rc = 0`, `instant_accepted = 1`). A subsequent attempt by staff to approve the earlier pending fallback booking is rejected cleanly with 51003.

9. Scenario 9: Advisory Maintenance Acknowledgement Repair (c13)

- *Objective:* Verify automatic repair and creation of advisory maintenance acknowledgement records during staff approval over spaces with active advisory notices.
- *Controlled Result:* `usp_booking_approve` detects an unacknowledged advisory maintenance notice on the space, approves the booking (`rc = 0`), and automatically inserts the missing row into `dbo.booking_advisory_acknowledgement`.

5.3.4 Comparison Results & Invariant Verification

Table 8 summarizes the comparative results across all test scenarios.

Test nario	Sce-	Concurrent Actions	Baseline (Raw DML)	Controlled (Task 12)
K1 (b01/c01)		Two instant submissions for same space & time	Trigger bypassed under RCSI; overlapping bookings committed ($Q_{BR1} = 1$)	Serialized via AppLock; winner <code>rc=0</code> , loser 51003, $Q_{BR1} = 0$
K2 (b02/c02)		Instant submit vs staff approval	Trigger bypassed under RCSI; overlapping booking committed ($Q_{BR1} = 1$)	Serialized; approval wins (<code>rc=0</code>), submit rejected (51003), $Q_{BR1} = 0$
K3 (b03/c03)		OOS escalation vs in-flight submit	Raw trigger Msg 50000 exception or booking leak	Serialized; escalation wins (<code>rc=0</code>), submit/approve rejected (51002)
T5/T7 (b05/c05)		Lock timeout & retry contract	Uncontrolled blocking for 18+ seconds	First contender times out with 51005 (5s); retry succeeds (<code>rc=0</code>)
K5 (b09/c09)		OOS ticket creation vs instant submit	Raw trigger Msg 50000 exception	Serialized; OOS ticket wins (<code>rc=0</code>), submit rejected (51002)
Staff (b10/c10)	Race	Two staff members approving overlapping pendings	Trigger bypassed under RCSI; both approvals commit ($Q_{BR1} = 1$)	Serialized; first approval wins (<code>rc=0</code>), second rejected (51003)
Soft (c11)	Gate	Instant submit with invalid purpose (<code>lecture</code>)	N/A	Falls back to pending (<code>rc=0</code> , <code>instant=0</code>) without auto-approval
Fallback (c12)	Race	Pending fallback vs new instant submit	N/A	Instant wins (<code>rc=0</code>); later approval of pending rejected (51003)
Ack (c13)	Repair	Staff approval under advisory maintenance	N/A	Approval succeeds (<code>rc=0</code>); auto-writes advisory ack record

Table 8: Comparative concurrency test results between Baseline (Raw DML) and Controlled (Task 12 Procedures)

The suite-wide invariant audit script (`audit_invariant.sql`) executed at the conclusion of the test suite confirmed:

```
1 PASS suite-audit: zero overlapping confirmed pairs AND zero confirmed-vs-OOS overlaps on
   TEST-13 spaces.
```

This empirically proves that the Task 12 application-lock protocol guarantees 100% data integrity ($Q_{BR1} = 0, Q_{NR6} = 0$) under concurrent execution.

6 Sample Data Generation

This section describes `14-data-generator` and the generated Phase 2 dataset. Our goal was not only to reach the minimum booking count, but also to keep the data believable enough so that Tasks 15 and 16 could measure real differences.

6.1 Scale and Time Coverage

The generator is deterministic and uses the seed 20260807. It writes one CSV per table, then `load.sql` imports them with `KEEPIDENTITY`, `CHECK_CONSTRAINTS`, and `BATCHSIZE=20000`. A small Mode-A step in the loader creates the maintenance impact history that the bulk path itself cannot produce. Table 9 summarizes the generated row counts and their coverage.

Table or data category	Rows	Coverage notes
Departments	8	Seeded master data for the campus units.
Users	2,200	Mix of students, lecturers, staff, and managers.
Spaces	120	Six space types across multiple buildings and floors.
Facilities	12	T14-named facilities used by the room finder.
Space_facilities	496	Realistic many-to-many facility distribution.
Maintenance	2,600	Both advisory and out-of-service records.
Bookings	120,000	Spans four academic years, from 2021-09-06 to 2025-06-21.
Booking approvals	78,178	Includes both instant and staff approvals.
Booking sessions	53,328	Covers check-in and completion history.
Advisory acknowledgements	44,163	Matches the confirmed bookings that overlap advisories.
Impact history	2	Created by the Mode-A update slice in <code>load.sql</code> .

Table 9: Generated Phase 2 data volume

6.2 Generation Strategy

The generator is single-threaded, so it owns the full booking schedule per space. That made it easier to keep confirmed bookings disjoint while still allowing pending, rejected, and cancelled requests to overlap like real demand. Booking times are spread across four academic years, with heavier activity near the start of each term. The same seed produced the same CSV hashes on a second run, so the dataset is easy to reproduce.

6.3 Exceptional Cases

The dataset includes advisory maintenance, out-of-service maintenance, escalation history, cancellations, no-shows, and advisory acknowledgements. The loader also keeps the migration-

owned `space_type_allowed_purpose` table untouched; it only verifies that the seeded reference data is already present. The verifier checked the main invariants, including no overlapping confirmed bookings, no confirmed booking over active out-of-service maintenance, no missing advisory acknowledgement, and the instant-approval purpose gate. All checks passed.

7 Analytical Queries

Phase 2 (project description §1.3) introduces four reporting queries that support the Facility Manager with accumulated booking and maintenance history: (1) total approved booking hours per space for a semester, (2) the number of approved bookings by weekday and hour for a semester, (3) available spaces satisfying a required capacity and facility list within a time period, and (4) approved bookings affected when a maintenance record is escalated to out-of-service. This section describes their implementation in `outputs/16-analytical-queries-G05.sql` and the results observed on the generated dataset.

Each query is wrapped in a stored procedure (`dbo.usp_task16_q1 ... dbo.usp_task16_q5`) so that the same query body can be executed repeatedly with different parameter sets, and each procedure validates its inputs (for example, it rejects a slot with `@slot_end <= @slot_start` with a descriptive `THROW`). The procedures share a small set of conventions derived from the Task 09 schema:

- *Confirmed bookings* are those with status `approved`, `checked_in`, or `completed`;
- soft-deleted rows (`is_deleted = 1`) are always excluded;
- time intervals are compared with the half-open overlap predicate `@start < row_end AND @end > row_start`;
- semester windows follow the U4 convention: Semester 1 is the half-open interval [September 1, February 1), excluded summer.

All results in this section were obtained on 2026-08-08 by executing the complete script against the Task 14 scratch database `CS486_G05_T14` (120,000 bookings); the raw evidence is recorded in `logs/eval/task16/`. The semester examples use Semester 1 of academic year 2024–2025, [2024-09-01, 2025-02-01). Table 10 introduces the stored procedures and their reporting purposes.

Stored procedure	Purpose
<code>usp_task16_q3_total_approved_hours_per_space</code>	Report 1: approved booking hours per space
<code>usp_task16_q4_weekday_hour_demand</code>	Report 2: approved bookings by weekday and hour
<code>usp_task16_q2_room_finder</code>	Report 3: available spaces with capacity and required facilities
<code>usp_task16_q5_escalation_impact</code>	Report 4: approved bookings affected by escalation
<code>usp_task16_q1_booking_conflict_check</code>	Booking-conflict check; read hint for the Task 15 tuning workload

Table 10: Analytical query inventory

The booking-conflict check is not one of the four reports: it shares its conflict semantics with the concurrency-safe write procedures of Task 12 and is analysed as the Task 15 tuning workload in Section 8. Reports 1–4 are detailed below, and the SQL excerpts are shown as stored procedures, matching the final Task 16 script.

7.1 Approved Booking Hours per Space (Report 1)

The first report answers: *how many confirmed booking hours did each space have in a semester?* All confirmed, non-deleted bookings whose requested interval overlaps the semester window are included, and each duration is clipped to the window so that a booking spilling over the semester boundary contributes only its in-semester hours. The aggregation groups by the stable space identity and orders by total hours descending so that the most loaded spaces appear first:

```

1 CREATE OR ALTER PROCEDURE dbo.usp_task16_q3_total_approved_hours_per_space
2     @semester_start DATETIME2,
3     @semester_end   DATETIME2
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     WITH semester_bookings AS (
9         SELECT
10             booking_id,
11             space_id,
12             CASE
13                 WHEN requested_start_time < @semester_start
14                     THEN @semester_start
15                 ELSE requested_start_time
16             END AS eff_start,
17             CASE
18                 WHEN requested_end_time > @semester_end
19                     THEN @semester_end
20                 ELSE requested_end_time
21             END AS eff_end
22         FROM dbo.bookings
23         WHERE is_deleted = 0
24             AND status IN ('approved', 'checked_in', 'completed')
25             AND requested_start_time < @semester_end
26             AND requested_end_time > @semester_start
27     )
28     SELECT
29         s.space_id,
30         s.space_code,
31         s.space_name,
32         s.capacity,
33         COUNT(sb.booking_id) AS confirmed_booking_count,
34         ROUND(
35             SUM(DATEDIFF_BIG(SECOND, sb.eff_start, sb.eff_end) / 3600.0),
36             2
37         ) AS total_hours_in_semester,
38         ROUND(
39             SUM(DATEDIFF_BIG(SECOND, sb.eff_start, sb.eff_end) / 3600.0)
40             / NULLIF(COUNT(sb.booking_id), 0),
41             2
42         ) AS avg_hours_per_booking
43     FROM dbo.spaces s
44     LEFT JOIN semester_bookings sb
45         ON sb.space_id = s.space_id
46     GROUP BY
47         s.space_id,
48         s.space_code,
49         s.space_name,
50         s.capacity
51     ORDER BY
52         total_hours_in_semester DESC,

```

```

53     s.space_code ASC;
54 END

```

Executed with @semester_start = '2024-09-01' and @semester_end = '2025-02-01', the query returned 120 spaces covering 901 confirmed booking intervals; Table 11 shows the top of the ordering.

Space	Cap.	Bookings	Hours	Avg hrs
Project Lab 131 (T14-BE-0032)	35	113	167.50	1.48
Project Lab 202 (T14-AL-0103)	40	111	164.00	1.48
Meeting Room 159 (T14-IN-0060)	30	100	147.50	1.48
Auditorium 200 (T14-LI-0101)	240	94	147.00	1.56
Computer Lab 165 (T14-IN-0066)	35	98	143.50	1.46
Classroom 117 (T14-IN-0018)	65	91	142.00	1.56

Table 11: Top spaces by total approved booking hours, Semester 1 AY2024–2025

The aggregation key is the surrogate `space_id`, so the grouping is stable even though the same space name recurs across buildings. The duration is derived from the *requested* interval, per the U4 decision, rather than the actual session duration.

7.2 Bookings by Weekday and Hour (Report 2)

This query characterises the demand pattern of a semester: *on which weekdays and hours do approved bookings start?* Knowing the weekday–hour distribution of confirmed booking starts lets the Facility Manager align staffing and opening hours with real demand. The weekday number must be meaningful regardless of the session's DATEFIRST, so it is computed with a pure integer expression anchored at 1900–01–01 and mapped with a modulo-7 arithmetic to 1 = Monday ... 7 = Sunday:

```

1 CREATE OR ALTER PROCEDURE dbo.usp_task16_q4_weekday_hour_demand
2     @semester_start DATETIME2,
3     @semester_end   DATETIME2
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     WITH semester_bookings AS (
9         SELECT booking_id, space_id, requested_start_time
10        FROM dbo.bookings
11        WHERE is_deleted = 0
12              AND status IN ('approved', 'checked_in', 'completed')
13              AND requested_start_time >= @semester_start
14              AND requested_start_time < @semester_end
15    )
16    SELECT
17        ((DATEDIFF(DAY, CONVERT(date, '19000101'),
18            CONVERT(date, sb.requested_start_time)) % 7) + 7) % 7 + 1
19        AS weekday_number,
20        CASE ((DATEDIFF(DAY, CONVERT(date, '19000101'),
21            CONVERT(date, sb.requested_start_time)) % 7) + 7) % 7 + 1
22            WHEN 1 THEN N'Monday'
23            WHEN 2 THEN N'Tuesday'
24            WHEN 3 THEN N'Wednesday'
25            WHEN 4 THEN N'Thursday'

```

```

26         WHEN 5 THEN N'Friday'
27         WHEN 6 THEN N'Saturday'
28         ELSE N'Sunday'
29     END AS weekday_name,
30     DATEPART(HOUR, sb.requested_start_time) AS start_hour,
31     COUNT(*) AS booking_count,
32     COUNT(DISTINCT sb.space_id) AS distinct_space_count
33 FROM semester_bookings sb
34 GROUP BY
35     ((DATEDIFF(DAY, CONVERT(date, '19000101'),
36         CONVERT(date, sb.requested_start_time)) % 7) + 7) % 7 + 1,
37     DATEPART(HOUR, sb.requested_start_time)
38 ORDER BY weekday_number, start_hour;
39 END

```

A half-open window is used: a booking contributes when its `requested_start_time` lies in `[@semester_start, @semester_end)`. With the same Semester 1 window the evaluation run produced 66 weekday/hour buckets, verified in the live evaluation; each bucket counts the confirmed bookings that start in that hour and the number of distinct spaces involved:

Column	Content
<code>weekday_number, weekday_name</code>	weekday of the booking start, Monday = 1
<code>start_hour</code>	hour of <code>requested_start_time</code> (0–23)
<code>booking_count</code>	number of confirmed bookings in the bucket
<code>distinct_space_count</code>	number of distinct spaces used in the bucket

Table 12: Weekday/hour bucket schema of Report 2

The row-level excerpt can be reproduced by re-running the procedure with the same window against the Task 14 database; the scheme of the result is stable, so the report can be regenerated for any semester without modifying the query body.

7.3 Available Spaces with Required Facilities (Report 3)

This report implements a room finder: *which spaces satisfy a requested capacity and every required facility during a given time period?* The caller passes the slot, the minimum capacity, and the list of required facilities as a JSON array, which the procedure normalises into a local table variable before evaluating the availability criteria. Three conditions must hold simultaneously for a space to appear in the result:

1. *Capacity and facility fit* — `capacity >= @minimum_capacity` and the space has *every* requested facility, verified by relational division (a double NOT EXISTS); a one-facility match is not sufficient;
2. *No booking conflict* — no confirmed, non-deleted booking of that space overlaps the proposed slot;
3. *No hard maintenance conflict* — no `open` or `in_progress` maintenance with impact level `out-of-service` overlaps the slot; advisory maintenance does *not* block availability.

The stored procedure is built around the following structure:

```

1 CREATE OR ALTER PROCEDURE dbo.usp_task16_q2_room_finder
2     @slot_start DATETIME2,
3     @slot_end DATETIME2,
4     @minimum_capacity INT,

```

```

5      @required_facilities_json NVARCHAR(MAX) = N'[]'
6  AS
7  BEGIN
8      SET NOCOUNT ON;
9
10 DECLARE @required_facilities TABLE (
11     facility_name NVARCHAR(255) PRIMARY KEY
12 );
13
14 INSERT INTO @required_facilities (facility_name)
15 SELECT DISTINCT CAST([value] AS NVARCHAR(255))
16 FROM OPENJSON(@required_facilities_json)
17 WHERE [value] IS NOT NULL
18     AND LTRIM(RTRIM(CAST([value] AS NVARCHAR(255)))) <> N'';
19
20 WITH booking_conflicts AS (
21     SELECT DISTINCT b.space_id
22     FROM dbo.bookings b
23     WHERE b.is_deleted = 0
24         AND b.status IN ('approved', 'checked_in', 'completed')
25         AND @slot_start < b.requested_end_time
26         AND @slot_end > b.requested_start_time
27 ),
28 maintenance_conflicts AS (
29     SELECT DISTINCT m.space_id
30     FROM dbo.maintenance m
31     WHERE m.is_deleted = 0
32         AND m.status IN ('open', 'in_progress')
33         AND m.impact_level = 'out-of-service'
34         AND m.start_time < @slot_end
35         AND (
36             m.completion_time IS NULL
37             OR m.completion_time > @slot_start
38         )
39 ),
40 facilities_ok AS (
41     SELECT s2.space_id
42     FROM dbo.spaces s2
43     WHERE NOT EXISTS (
44         SELECT 1
45         FROM @required_facilities rf
46         WHERE NOT EXISTS (
47             SELECT 1
48             FROM dbo.space_facilities sf
49             INNER JOIN dbo.facilities f
50                 ON f.facility_id = sf.facility_id
51             WHERE sf.space_id = s2.space_id
52                 AND f.name = rf.facility_name
53         )
54     )
55 )
56 SELECT
57     s.space_id,
58     s.space_code,
59     s.space_name,
60     s.space_type,
61     s.capacity,
62     s.current_status AS status_hint,
63     STRING_AGG(f.name, ', ') AS facility_summary
64 FROM dbo.spaces s

```



```

65 LEFT JOIN dbo.space_facilities sf
66     ON sf.space_id = s.space_id
67 LEFT JOIN dbo.facilities f
68     ON f.facility_id = sf.facility_id
69 LEFT JOIN booking_conflicts bc
70     ON bc.space_id = s.space_id
71 LEFT JOIN maintenance_conflicts mc
72     ON mc.space_id = s.space_id
73 WHERE s.capacity >= @minimum_capacity
74     AND s.current_status NOT IN ('retired', 'temporarily_closed')
75     AND bc.space_id IS NULL
76     AND mc.space_id IS NULL
77     AND EXISTS (
78         SELECT 1
79         FROM facilities_ok fo
80         WHERE fo.space_id = s.space_id
81     )
82 GROUP BY
83     s.space_id,
84     s.space_code,
85     s.space_name,
86     s.space_type,
87     s.capacity,
88     s.current_status
89 ORDER BY
90     s.capacity ASC,
91     s.space_code ASC;
92 END

```

The facility summary aggregates every facility of a qualifying space with `STRING_AGG`, so the caller sees exactly what is installed. Executed with `@slot_start = '2024-09-16T10:00:00'`, `@slot_end = '2024-09-16T12:00:00'`, `@minimum_capacity = 20` and the two-facility list `N'["T14 Projector","T14 Whiteboard"]'`, the query found exactly one available space (as expected from the multi-facility matching): Computer Lab 165 (T14-IN-0066, space 205065) with capacity 35, status `available`, and the facility summary shown in Table 13. With the empty facility list the same query returns all capacities and locations free of booking and maintenance conflicts.

Property	Value
Space	205065 — Computer Lab 165 (T14-IN-0066)
Type / capacity	computer_lab / 35
Status hint	available
Facility list	T14 Desktop Computer x3, T14 Livestreaming Kit x1, T14 Presenter Remote x4, T14 Projector x4, T14 Video Conference Unit x3, T14 Whiteboard x1
Requested window	2024-09-16 10:00–12:00

Table 13: Room-finder result for a two-facility search

7.4 Approved Bookings Affected by Escalation (Report 4)

The last report supports business rule NR4. When the impact level of an *advisory* maintenance record is escalated to `out-of-service`, all already-approved bookings that overlap the

maintenance period must be identified so that staff can contact the requesters. The query therefore operates on maintenance *history* events rather than on the current impact level of the maintenance record: it filters `maintenance_impact_history` for rows with `prior_level = 'advisory'` and `new_level = 'out-of-service'`, joins those events to the affected maintenance record, to the confirmed bookings overlapping the maintenance interval, and to the advisory acknowledgement recorded when the booking was submitted:

```

1 CREATE OR ALTER PROCEDURE dbo.usp_task16_q5_escalation_impact
2     @from_escalation_time DATETIME2 = NULL,
3     @to_escalation_time   DATETIME2 = NULL,
4     @maintenance_id       INT = NULL
5 AS
6 BEGIN
7     SET NOCOUNT ON;
8
9     WITH escalation_events AS (
10        SELECT
11            mih.history_id,
12            mih.maintenance_id,
13            mih.changed_at AS escalation_time,
14            mih.changed_by,
15            mih.prior_level,
16            mih.new_level,
17            mih.reason AS escalation_reason,
18            m.space_id,
19            m.problem_description,
20            m.start_time AS maintenance_start_time,
21            m.completion_time AS maintenance_end_time
22        FROM dbo.maintenance_impact_history mih
23        INNER JOIN dbo.maintenance m
24            ON m.maintenance_id = mih.maintenance_id
25        WHERE mih.prior_level = 'advisory'
26            AND mih.new_level = 'out-of-service'
27            AND m.is_deleted = 0
28            AND (
29                @from_escalation_time IS NULL
30                OR mih.changed_at >= @from_escalation_time
31            )
32            AND (
33                @to_escalation_time IS NULL
34                OR mih.changed_at < @to_escalation_time
35            )
36            AND (
37                @maintenance_id IS NULL
38                OR mih.maintenance_id = @maintenance_id
39            )
40    )
41    SELECT
42        ee.escalation_time,
43        escalator.full_name AS escalated_by,
44        ee.new_level,
45        s.space_id,
46        s.space_code,
47        s.space_name,
48        b.booking_id,
49        b.requested_start_time,
50        b.requested_end_time,
51        b.purpose,
52        b.status AS booking_status,

```

```

53     requester.full_name AS requester_name,
54     requester.email AS requester_email,
55     baa.acknowledged_at,
56     ROUND(
57         DATEDIFF_BIG(
58             SECOND,
59             CASE
60                 WHEN b.requested_start_time < ee.maintenance_start_time
61                     THEN ee.maintenance_start_time
62                 ELSE b.requested_start_time
63             END,
64             CASE
65                 WHEN b.requested_end_time > COALESCE(
66                     ee.maintenance_end_time, b.requested_end_time
67                 )
68                 THEN COALESCE(
69                     ee.maintenance_end_time, b.requested_end_time
70                 )
71                 ELSE b.requested_end_time
72             END
73         ) / 3600.0,
74         2
75     ) AS overlap_hours
76 FROM escalation_events ee
77 INNER JOIN dbo.booking_approvals ba
78     ON ba.decision = 'approved'
79     AND ba.decision_time <= ee.escalation_time
80 INNER JOIN dbo.bookings b
81     ON b.booking_id = ba.booking_id
82     AND b.space_id = ee.space_id
83     AND b.is_deleted = 0
84     AND b.status IN ('approved', 'checked_in', 'completed')
85     AND b.requested_start_time < COALESCE(
86         ee.maintenance_end_time,
87         CONVERT(DATETIME2, '9999-12-31T23:59:59')
88     )
89     AND b.requested_end_time > ee.maintenance_start_time
90 INNER JOIN dbo.booking_advisory_acknowledgement baa
91     ON baa.booking_id = b.booking_id
92     AND baa.maintenance_id = ee.maintenance_id
93 INNER JOIN dbo.spaces s
94     ON s.space_id = ee.space_id
95 INNER JOIN dbo.users requester
96     ON requester.user_id = b.requester_id
97 INNER JOIN dbo.users escalator
98     ON escalator.user_id = ee.changed_by
99 ORDER BY
100     ee.escalation_time DESC,
101     b.requested_start_time ASC,
102     b.booking_id ASC;
103 END

```

The dependency on the acknowledgement join is deliberate: a booking can be reported only if the requester was informed of the advisory at submission time, which is exactly the population that NR4 wants contacted. For each row the procedure also computes `overlap_hours`, the portion of the booking interval that falls inside the maintenance period, so that staff can reprioritise the follow-up by impact size. The query is strictly read-only: it never mutates or cancels bookings (the concurrency-safe cancellation procedures of Task 12 are the only writers in that domain).

Executed with all-NULL filters (i.e., all escalation history), the live run returned 655 affected confirmed bookings. Table 14 shows the first rows of the result, referring to the escalation of maintenance 207478 (“Two ceiling lights flicker intermittently”, Meeting Room 110):

Hist. id	Escalated at	By	Maint id	Booking	Requester
1	2026-08-07 16:55:56	Lan Hoang	207478	368665	Vinh Do
1	2026-08-07 16:55:56	Lan Hoang	207478	321574	Son Nguyen
1	2026-08-07 16:55:56	Lan Hoang	207478	386469	Trang Tran

Table 14: Escalation-affected bookings (excerpt of the 655 returned rows)

Each row additionally carries the requested time window, purpose, booking status, requester email and phone, the acknowledgement timestamp, and the computed `overlap_hours`; those columns are condensed in the excerpt above.

7.5 Work Assignment

The four reports were implemented by the group members listed below, each student contributing one query to the shared script:

Student	Report	Primary contribution
Phạm Hữu Nam (24125015)	Report 1	Semester-aggregated approved hours with the U4 window
Cao Quảng Hưng (24125010)	Report 2	Deterministic Monday=1 weekday grid
Trần Đình Quốc Thắng (24125079)	Report 3	JSON facility list with relational division
Nguyễn Hữu Phước (24125018)	Report 4	Escalation-history join with approval and acknowledgement

Table 15: Individual analytical-query contributions

8 Indexing & Query Tuning

This section summarizes `15-index-tuning-report`. We tested the two required workloads, Q1 booking-conflict check and Q2 room finder, and then selected Q3 and Q5 as the two reporting queries because they had the clearest logical-read cost on the generated Task 14 dataset. The indexes were created only in the scratch database for measurement; they were not added to the production migration script.

8.1 Benchmark Setup

The benchmark used the Task 14 database `CS486_G05_T14`, which contains 120,000 bookings from 2021-09-06 to 2025-06-21. Each query was measured three times before indexing and three times after indexing, then the median value was reported. Table 16 lists the dataset, parameters, and measurement protocol used for the benchmark.

Item	Value used in Task 15
Database and server	CS486_G05_T14 on LAPTOP-J7QNDDF0\SQL1, SQL Server 2022 16.0.1190.2.
Dataset	120,000 bookings, 120 spaces, 2,600 maintenance records, and 44,163 advisory acknowledgements from Task 14.
Cache and statistics protocol	DBCC FREEPROCCACHE, DBCC DROPCLEANBUFFERS, SET STATISTICS IO ON, SET STATISTICS TIME ON, and estimated ShowPlan XML capture.
Q1 parameter	space_id = 205111, 2021-09-06 11:00-12:00.
Q2 parameter	2024-09-16 10:00-12:00, minimum capacity 20, required facilities T14 Projector and T14 Whiteboard.
Q3 parameter	Semester 1 AY2024-2025, [2024-09-01, 2025-02-01).
Q5 parameter	All escalation-history events present in the Task 14 load.

Table 16: Task 15 benchmark setup

8.2 Selected Workloads and Plan Evidence

Q1 and Q2 were mandatory because they support the main availability path. From the report queries, we selected Q3 and Q5 instead of Q4.

Q3 scans the semester booking history. Q5 joins escalation history with approvals, bookings, acknowledgements, spaces, and users. In the baseline run, Q3 used 1,269 user-table logical reads and Q5 used 16,100, so they were the better tuning targets.

Estimated plan files were saved in the Task 15 plan folder. The main change was filtered seeks on Q1 to Q3, plus a better booking-overlap path on Q5.

8.3 Before-and-After Results

Table 17 compares the median logical reads and elapsed time before and after applying each candidate index. Lower values indicate reduced I/O and faster query execution. The final column summarizes whether each index should be adopted, retained for shared benefits, or deferred until a larger dataset provides stronger evidence.

Query	Candidate index	Before reads	After reads	Elapsed time	Decision
Q1	Booking cover index	23	14	6 ms → 3 ms	Adopt.
Q2	Booking time-window index + maintenance time-window index	2,058	1,239	90 ms → 15 ms	Adopt maintenance; keep booking index because Q3 benefits from it.
Q3	Booking time-window index	1,269	244	51 ms → 20 ms	Adopt.
Q5	Booking cover index + approval-time candidate	16,100	12,787	220 ms → 187 ms	Adopt shared booking index; defer approval-time index.

Table 17: Query-tuning results on the generated dataset

The clearest result was Q3: the filtered confirmed-booking time-window index reduced booking reads from 1,263 to 238 and cut median elapsed time by 60.8%. For Q2, the maintenance index was the main reason the query improved; the booking time-window index read slightly more pages for that exact room-finder parameter, but it stayed in the final list because it strongly helped Q3. Q5 also improved, but mostly from the shared booking index. The approval-time candidate looked reasonable on paper, but the Task 14 history table was too small to prove a real benefit.

8.4 Final Recommendation

Based on the measured runs, our recommended adoption list is:

- Booking cover index on confirmed bookings;
- Booking time-window index for semester scans;
- Maintenance time-window index for active out-of-service checks.

The approval-time candidate should be kept as a documented idea, but not adopted yet. It should be re-tested when the system has a much larger approval-history table.

9 Normalization Validation

The relational schema was re-validated after the Phase 2 redesign (09-updated-erd-and-logical-design, §5). The check covers every new or changed relation—`maintenance`, `maintenance_impact_history`, `booking_advisory_acknowledgement`, `booking_approvals`, `users`, `spaces`, and `space_type_allowed_purpose`—and verifies the first, second, and third normal forms.

9.1 Functional Dependencies

Table 18 lists, for each affected relation, the non-trivial functional dependencies and the candidate keys. All attributes of a relation depend functionally on every candidate key; the table

states the minimal sets from which all other attributes are derived and distinguishes candidate keys from non-key determinants.

Relation	Functional dependencies	Candidate keys
<code>maintenance</code>	<code>maintenance_id</code> $\rightarrow$ all attributes, including <code>impact_level</code>	<code>maintenance_id</code>
<code>maintenance_impact_history</code>	<code>history_id</code> $\rightarrow$ all attributes	<code>history_id</code>
<code>booking_advisory_acknowledgement</code>	<code>ack_id</code> $\rightarrow$ all attributes; (<code>booking_id</code> , <code>maintenance_id</code>) $\rightarrow$ all attributes	<code>ack_id</code> ; (<code>booking_id</code> , <code>maintenance_id</code>)
<code>booking_approvals</code>	<code>approval_id</code> $\rightarrow$ all attributes; <code>booking_id</code> $\rightarrow$ all attributes. The instant/staff pathway is derived from <code>approver_id = -1</code>	<code>approval_id</code> ; <code>booking_id</code>
<code>users</code>	<code>user_id</code> $\rightarrow$ all attributes (unchanged from Phase 1)	<code>user_id</code>
<code>spaces</code>	<code>space_id</code> $\rightarrow$ all attributes (same key as in Phase 1)	<code>space_id</code>
<code>space_type_allowed_purpose</code>	(<code>space_type</code> , <code>purpose</code>) $\rightarrow$ <code>created_at</code> , <code>updated_at</code>	(<code>space_type</code> , <code>purpose</code>)

Table 18: Functional dependencies in Phase 2 relations

9.2 Third Normal Form Validation

Every relation satisfies the first normal form: all attributes are atomic and no repeating groups exist. The second normal form holds because no partial dependency on any candidate key exists. In `booking_advisory_acknowledgement` the composite unique key (`booking_id`, `maintenance_id`) is a *full* alternate key: neither `booking_id` alone nor `maintenance_id` alone determines any other attribute. In `space_type_allowed_purpose` the composite primary key is likewise full, so no proper subset determines the non-key columns. All other relations use a single-column primary key, for which a partial dependency is impossible.

For the third normal form, every non-trivial dependency has a candidate key as its determinant: the non-key attributes of each relation depend only on the candidate key or keys listed in Table 18, and no non-key attribute depends on another non-key attribute, so no transitive dependency exists.

The review found no violation in the final schema. In particular, `booking_approvals` holds no stored instant/staff origin: the pathway is derived from the reserved system approver (`approver_id = -1`), so the relation carries no non-key dependency and remains in 3NF (and BCNF). All seven affected relations therefore remain in at least third normal form after the Phase 2 changes.

10 Conclusion

Phase 2 transformed the Campus Space Management System into a scalable and concurrency-safe database solution. The revised design supports instant bookings, maintenance escalation, soft-gate fallbacks, and advisory notices while preserving the core Phase 1 business rules. Non-destructive schema migration maintained compatibility with existing data, and normalization ensured a clear and consistent relational structure. Concurrency control using SQL Server application locks eliminated critical booking race conditions and preserved the required data invariants during parallel testing. Tests on more than 100,000 synthetic bookings also demonstrated that targeted indexes can substantially improve analytical-query performance. Overall, the project provides a reliable foundation for campus-space operations. Future work may introduce semester-based table partitioning and finer per-space-day lock granularity if production workloads reveal significant archival or contention bottlenecks.